

LIBERTAS

Curso: Sistemas de Informação

Disciplina: Estrutura de Dados II

Professor: MSc. Jonathan Henrique Jeremias Souza

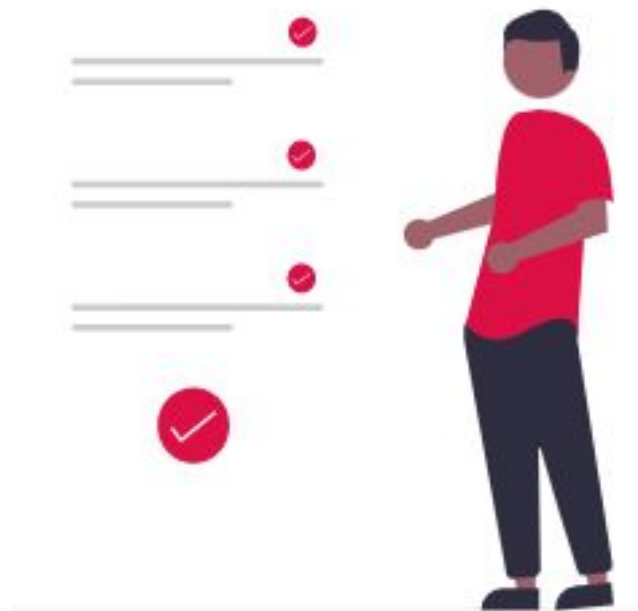
E-mail: jonathansouza@libertas.edu.br



Libertas[®]
FACULDADES INTEGRADAS

AGENDA

- Recursividade;



RECURSIVIDADE

Introdução

- Fundamental em Matemática e Ciência da Computação
 - Um programa recursivo é um programa que chama a si mesmo
 - Uma função recursiva é definida em termos dela mesma
- Exemplos
 - Números naturais, Função fatorial, Árvore
- Conceito poderoso
 - Define conjuntos infinitos com comandos finitos

RECURSIVIDADE

Introdução

- A recursividade é uma estratégia que pode ser utilizada sempre que o cálculo de uma função para o valor n , pode ser descrita a partir do cálculo desta mesma função para o termo anterior $(n-1)$.

Exemplo – Função fatorial:

$$n! = n * (n-1) * (n-2) * (n-3) * \dots * 1$$

$$(n-1)! = (n-1) * (n-2) * (n-3) * \dots * 1$$

logo:

$$n! = n * (n-1)!$$

RECURSIVIDADE

Recursividade

- Definição: dentro do corpo de uma função, chamar novamente a própria função
 - recursão direta: a função A chama a própria função A
 - recursão indireta: a função A chama uma função B que, por sua vez, chama A

RECURSIVIDADE

Condição de parada

- Nenhum programa nem função pode ser exclusivamente definido por si
 - Um programa seria um loop infinito
 - Uma função teria definição circular
- Condição de parada
 - Permite que o procedimento pare de se executar
 - $F(x) > 0$ onde x é decrescente
- Objetivo
 - Estudar recursividade como ferramenta prática!

RECURSIVIDADE

Recursividade

- Para cada chamada de uma função, recursiva ou não, os parâmetros e as variáveis locais são empilhados na pilha de execução.

RECURSIVIDADE

Execução

- Internamente, quando qualquer chamada de função é feita dentro de um programa, é criado um **Registro de Ativação** na **Pilha de Execução** do programa
- O registro de ativação armazena os parâmetros e variáveis locais da função bem como o “ponto de retorno” no programa ou subprograma que chamou essa função.
- Ao final da execução dessa função, o registro é desempilhado e a execução volta ao subprograma que chamou a função.

Exemplo

```
public class Fatorial {  
  
    static int fat1(int n) {  
        int r;  
        if (n <= 0)  
            r = 1;  
        else  
            r = n * fat1(n - 1);  
        return r;  
    }  
  
    static int fat2(int n) {  
        if (n <= 0)  
            return 1;  
        else  
            return n * fat2(n - 1);  
    }  
}
```

```
public static void main(String[] args) {  
    int f;  
  
    f = fat1(4);  
    System.out.println(f);  
  
    f = fat2(4);  
    System.out.println(f);  
}
```

RECURSIVIDADE

Execução

- A complexidade de tempo do fatorial recursivo é $O(n)$.
- Mas a complexidade de espaço também é $O(n)$, devido a pilha de execução
- Já no fatorial não recursivo a complexidade de espaço é $O(1)$

```
public class Fatorial2 {  
  
    static int fat(int n) {  
        int f = 1;  
  
        while (n > 0) {  
            f = f * n;  
            n = n - 1;  
        }  
        return f;  
    }  
  
    public static void main(String[] args) {  
        int resultado = fat(4);  
        System.out.println(resultado);  
    }  
}
```

RECURSIVIDADE

Recursividade

- Portanto, a recursividade nem sempre é a melhor solução, mesmo quando a definição matemática do problema é feita em termos recursivos.

RECURSIVIDADE

Fibonacci

- Outro exemplo: Série de Fibonacci:
 - $F_n = F_{n-1} + F_{n-2}$ $n > 2$,
 - $F_0 = 0$ $F_1 = 1$
 - 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89...

```
public class Fibonacci {  
  
    static int fib(int n) {  
        if (n <= 0)  
            return 0;  
        else if (n == 1)  
            return 1;  
        else  
            return fib(n - 1) + fib(n - 2);  
    }  
  
    public static void main(String[] args) {  
        int resultado = fib(6);  
        System.out.println(resultado);  
    }  
}
```

RECURSIVIDADE

Análise da função Fibonacci

- Ineficiência em Fibonacci
 - Termos F_{n-1} e F_{n-2} são computados independentemente
 - Número de chamadas recursivas = número de Fibonacci!
 - Custo para cálculo de F_n
 - $O(n)$ onde $= (1 + 5)/2 = 1,61803...$
 - Golden ratio
 - Exponencial!!!

RECURSIVIDADE

Fibonacci não recursivo

- Complexidade: $O(n)$
- Conclusão: não usar recursividade cegamente!

```
public class Fibonacci2 {  
  
    static int fiblter(int n) {  
        int i = 1, F = 0;  
  
        for (int k = 1; k <= n; k++) {  
            F = F + i;  
            i = F - i;  
        }  
  
        return F;  
    }  
  
    public static void main(String[] args) {  
        int resultado = fiblter(6);  
        System.out.println(resultado);  
    }  
}
```

RECURSIVIDADE

Quando vale a pena usar recursividade

- Recursividade vale a pena para Algoritmos complexos, cuja a implementação iterativa é complexa e normalmente requer o uso explícito de uma pilha
- Dividir para Conquistar (Ex. Quicksort)
- Caminhamento em Árvores (pesquisa, backtracking)

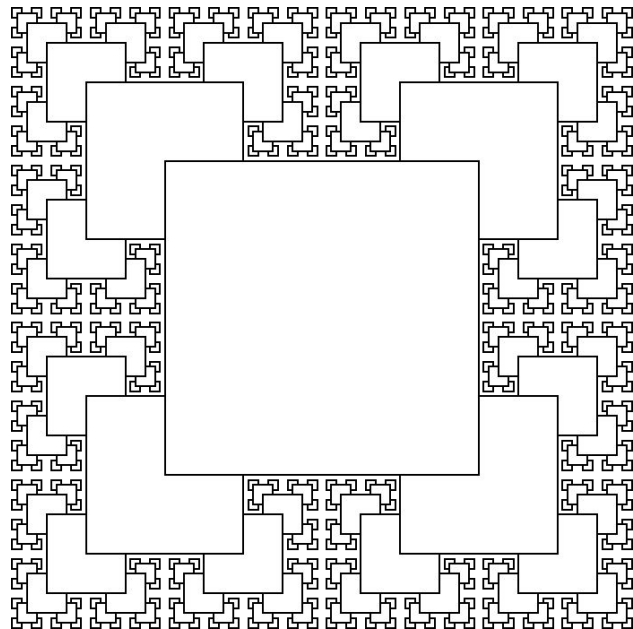
RECURSIVIDADE

Dividir para Conquistar

- Duas chamadas recursivas
 - Cada uma resolvendo a metade do problema
- Muito usado na prática
 - Solução eficiente de problemas
 - Decomposição
- Não se reduz trivialmente como fatorial
 - Duas chamadas recursivas
- Não produz recomputação excessiva como fibonacci
 - Porções diferentes do problema

RECURSIVIDADE

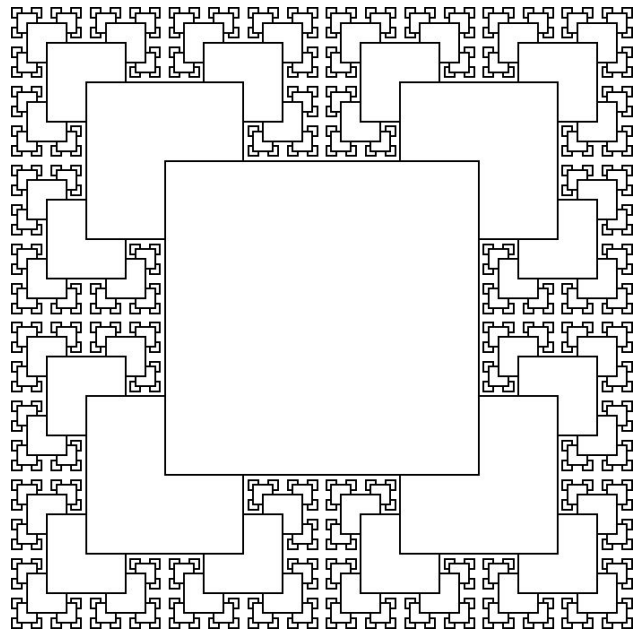
Outros exemplos de recursividade



```
public class Estrela {  
    static void estrela(int x, int y, int r) {  
        if (r > 0) {  
            estrela(x - r, y + r, r / 2);  
            estrela(x + r, y + r, r / 2);  
            estrela(x - r, y - r, r / 2);  
            estrela(x + r, y - r, r / 2);  
  
            box(x, y, r);  
        }  
    }  
    static void box(int x, int y, int r) {  
        System.out.println("Box em (" + x + "," + y + ") com r=" + r);  
    }  
  
    public static void main(String[] args) {  
        estrela(0, 0, 8);  
    }  
}
```

RECURSIVIDADE

Outra implementação: estrela



```
public class Estrela {  
    static void estrela(int x, int y, int r) {  
        if (r > 1) {  
            estrela(x - r, y + r, r / 2);  
            estrela(x + r, y + r, r / 2);  
            estrela(x - r, y - r, r / 2);  
            estrela(x + r, y - r, r / 2);  
        }  
        box(x, y, r);  
    }  
  
    static void box(int x, int y, int r) {  
        System.out.println("Box em (" + x + "," + y + ") com r=" + r);  
    }  
  
    public static void main(String[] args) {  
        estrela(0, 0, 8);  
    }  
}
```

RECURSIVIDADE

Exemplo simples: régua

```
public class Regua {  
    static void regua(int l, int r, int h) {  
        if (h > 0) {  
            int m = (l + r) / 2;  
  
            marca(m, h);  
  
            regua(l, m, h - 1);  
            regua(m, r, h - 1);  
        }  
    }  
}
```

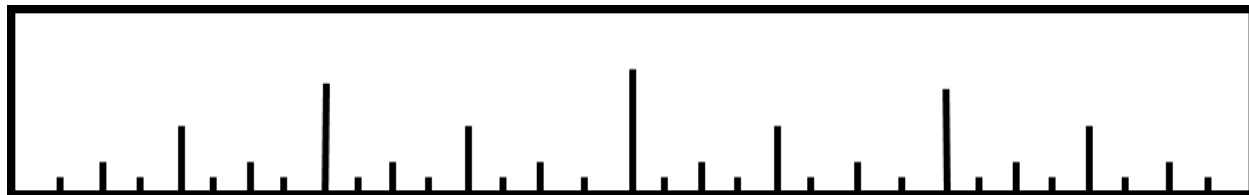
```
static void marca(int pos, int h) {  
    System.out.println("Marca em " + pos + " altura " + h);  
}
```

```
public static void main(String[] args) {  
    regua(0, 16, 4);  
}
```

l → onde começo (lado left, esquerdo)

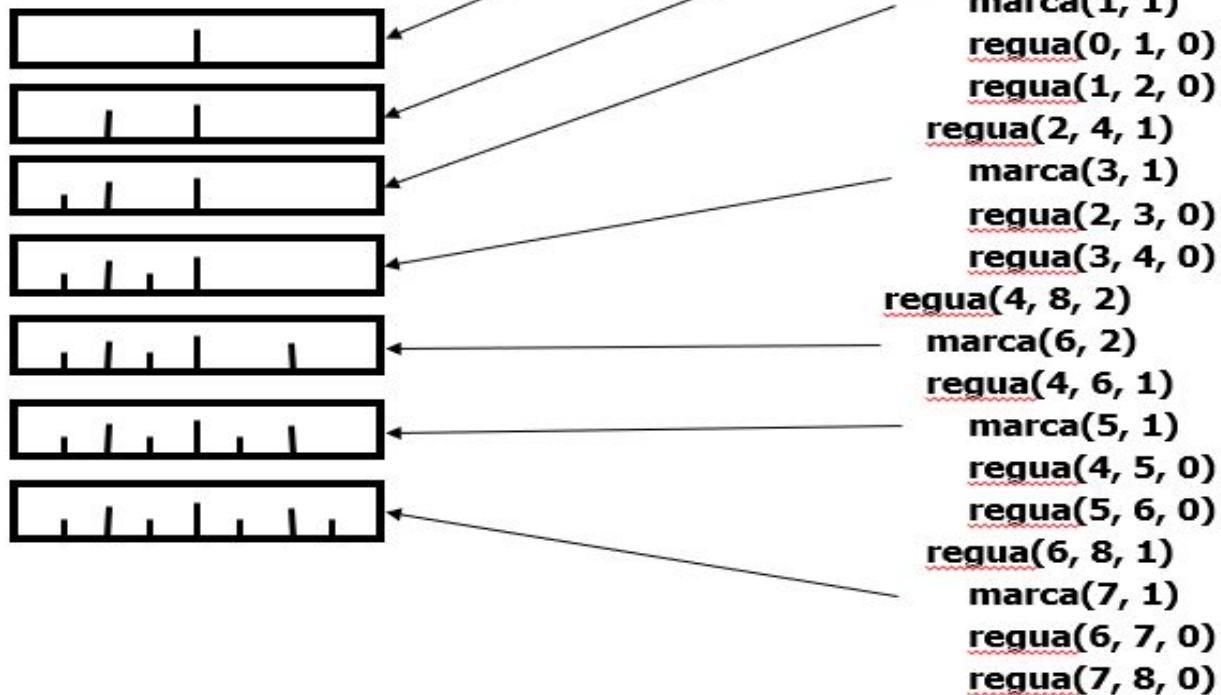
r → onde termino (lado right, direito)

h → qual o tamanho da marca que vou desenhar (**h**eight, altura)



RECURSIVIDADE

Exemplo simples: régua

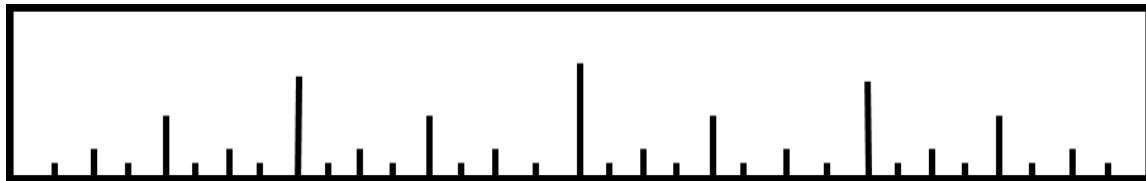


RECURSIVIDADE

Outra implementação: régua

```
public class Regua {  
  
    static void regua2(int l, int r, int h) {  
        int m = (l + r) / 2;  
  
        marca(m, h);  
  
        if (h > 1) {  
            regua2(l, m, h - 1);  
            regua2(m, r, h - 1);  
        }  
    }  
}
```

```
static void marca(int pos, int h) {  
    System.out.println("Marca em " + pos + " altura " + h);  
}  
  
public static void main(String[] args) {  
    regua2(0, 16, 4);  
}
```



RECURSIVIDADE

Exercício

- Crie uma função recursiva que calcula a potência de um número:
 - Como escrever a função para o termo n em função do termo anterior?
 - Qual a condição de parada?

RECURSIVIDADE

Exercício

```
public class Calculadora {  
  
    public static int pot(int base, int exp) {  
        // Condição de parada (Caso Base)  
        if (exp == 0) {  
            return 1;  
        }  
        // Termo n em função do termo anterior (Passo Recursivo)  
        return base * pot(base, exp - 1);  
    }  
  
    public static void main(String[] args) {  
        int resultado = pot(2, 3);  
        System.out.println("O resultado de 2 elevado a 3 é: " + resultado);  
    }  
}
```

MUITO OBRIGADO!

Ficou com alguma
dúvida?



MSc. Jonathan Henrique Jeremias Souza
jonathansouza@libertas.edu.br
www.libertas.edu.br



Libertas[®]
FACULDADES INTEGRADAS

Referências Bibliográficas

- ASCENCIO, A.F.G; ARAÚJO, G. S.; Estruturas de dados: algoritmos, análise da complexidade e implementações em JAVA e C/C++. São Paulo: Pearson, 2010
- PUGA, S.; RISSETTI, G.; Lógica de programação e estruturas de dados com aplicações em Java. 2ª Ed.; Pearson, 2009
- TENENBAUM, Aaron M.; LANGSAM, Yedidyah; AUGENSTEIN, Moshe J.; Estruturas de Dados Usando C. São Paulo: Pearson, 2009
- Estrutura de dados/ José Marcio Benite Ramos; Liluyoud Cury de Lacerda; Sara Luize Oliveira Duarte; org. Instituto Federal de Educação, Ciência e Tecnologia; Universidade Federal do Mato Grosso - Cuiabá: UFMT; Porto Velho: IFRO, 2013.
- David Menotti: Algoritmos e Estruturas de Dados I - Aula 6 - Recursividade (DECOM – UFOP);